

Module 1 Note: React Fundamentals

Components, List Rendering, Form Handling & React Context API

Topic: Web Development / React

Document Version: 1.0

01 React Components

Components are the foundational building blocks of any React application. A component is a self-contained, reusable piece of UI that manages its own rendering, state, and logic.

- **Functional Components:** Modern JavaScript functions that return JSX (JavaScript XML) describing what should appear on the screen.
- **Props (Properties):** Read-only inputs passed from parent components to child components to dynamicize content.

```
// Define a functional component accepting props
function UserCard({ name, role }) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>Role: {role}</p>
    </div>
  );
}
```

02 Rendering Lists in React

To render multiple dynamic elements in React, we use the standard JavaScript `Array.prototype.map()` function to transform data arrays into arrays of JSX elements.

Key Rule: The `key` Prop

Always assign a unique key prop (preferably a unique ID) to the top-level element inside the `map()` call. This allows React to identify modified, added, or removed items efficiently during re-renders.

```
function UserList() {
  const users = [
    { id: 101, name: 'Alice', role: 'Developer' },
    { id: 102, name: 'Bob', role: 'Designer' },
    { id: 103, name: 'Charlie', role: 'Product Owner' }
  ];

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>
          <strong>{user.name}</strong> - {user.role}
        </li>
      ))}
    </ul>
  );
}
```

```

        </li>
      )}
    </ul>
  );
}

```

03 Handling Forms in React

Forms in React are typically implemented using **Controlled Components**, where form input values are bound directly to React state using the `useState` hook.

- `value` attribute links the input element to state.
- `onChange` listener updates state as the user types.
- `onSubmit` captures submission and uses `e.preventDefault()` to avoid traditional page reloads.

```

import React, { useState } from 'react';

function LoginForm() {
  const [email, setEmail] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Submitted: ${email}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>Email:</label>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
      <button type="submit">Login</button>
    </form>
  );
}

```

04 React Context API

The **React Context API** solves the problem of "*prop drilling*" (passing data manually down through many nested component levels). It provides a way to share values globally across the component tree.

3 Core Steps to Use Context:

1. **Create Context:** Use `React.createContext()`.

2. **Provide Context:** Wrap child components with `<MyContext.Provider value={...}>`.
3. **Consume Context:** Retrieve values using the `useContext(MyContext)` hook.

```
import React, { createContext, useContext, useState } from 'react';

// 1. Create Context
const ThemeContext = createContext();

// 2. Provider Component
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme((prev) => (prev === 'light' ? 'dark' : 'light'));
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Consumer Component
function ThemeButton() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  return (
    <button onClick={toggleTheme}>
      Current Theme: {theme}
    </button>
  );
}
```